CSCC69 Operating Systems

What we have seen so far

Abstraction & virtualization:

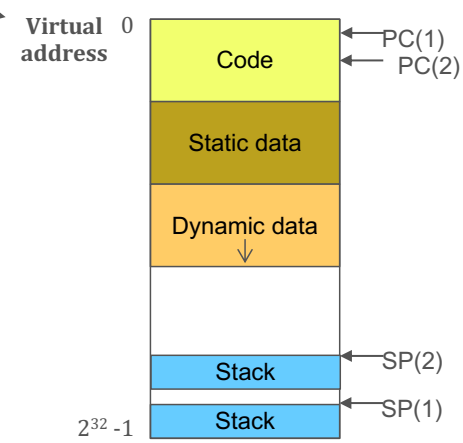
- Processes, threads
- Virtualizing CPUs (scheduling)
- Virtual memory (address space abstraction)

- How does the OS decide where in physical memory the address space is located?
- Binaries have virtual addresses hard-coded. How are they translated to actual physical addresses?

```
push   ebp
mov    ebp,esp
sub    esp,0x8
sub    esp,0xc
push   0x80484d0
call   80482e0 <printf@plt>
add    esp,0x10
nop
leave
ret


lea    ecx,[esp+0x4]
and    esp,0xfffffff0
push   DWORD PTR [ecx-0x4]
push   ebp
mov    ebp,esp
push   ecx
sub    esp,0x4
call   804840b <foo>
```

Virtual addresses

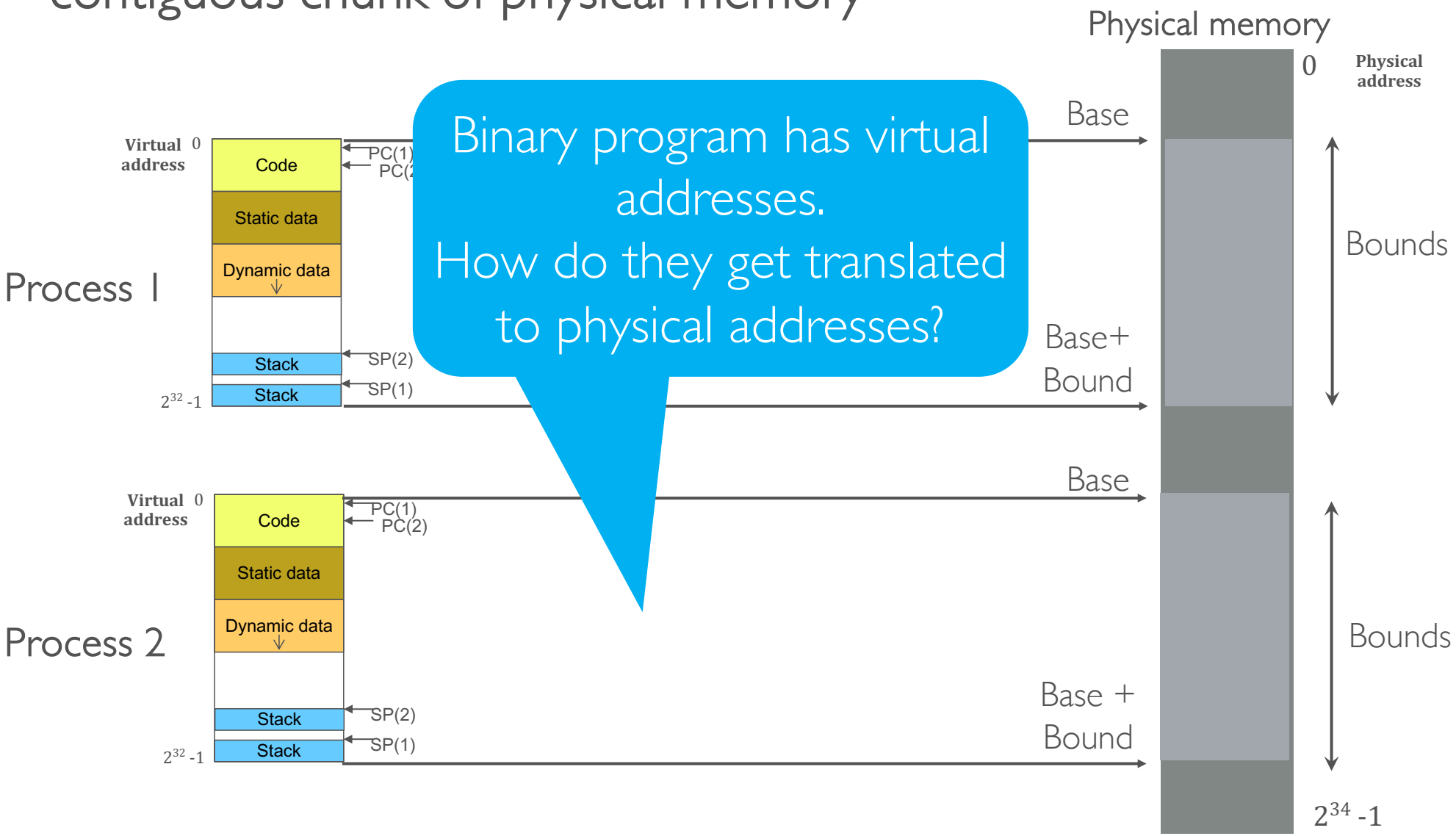


Virtual Memory Goals

- Provide a convenient abstraction for programming by giving each program its own virtual address space
- Enforce protection by preventing one process from messing with another's memory
- Allow programs to see more memory than exists
- Allocate scarce memory resources among competing processes to maximize performance with minimal overhead

Attempt 1: Base and bound registers

Attempt 1: Map address space of each process to a contiguous chunk of physical memory



Binary program has virtual addresses.
How do they get translated to physical addresses?

Base & Bound registers

Two special privileged registers : `base` and `bound`

On each load/store/jump

- **Physical address** = **virtual address** + `base`
- Check $0 \leq$ **virtual address** < `bounds`, else trap to kernel

Who does the above calculation (address translation)?
Who sets the base and bounds registers?

Who should control Base & Bound registers?

What entity should do address translation?

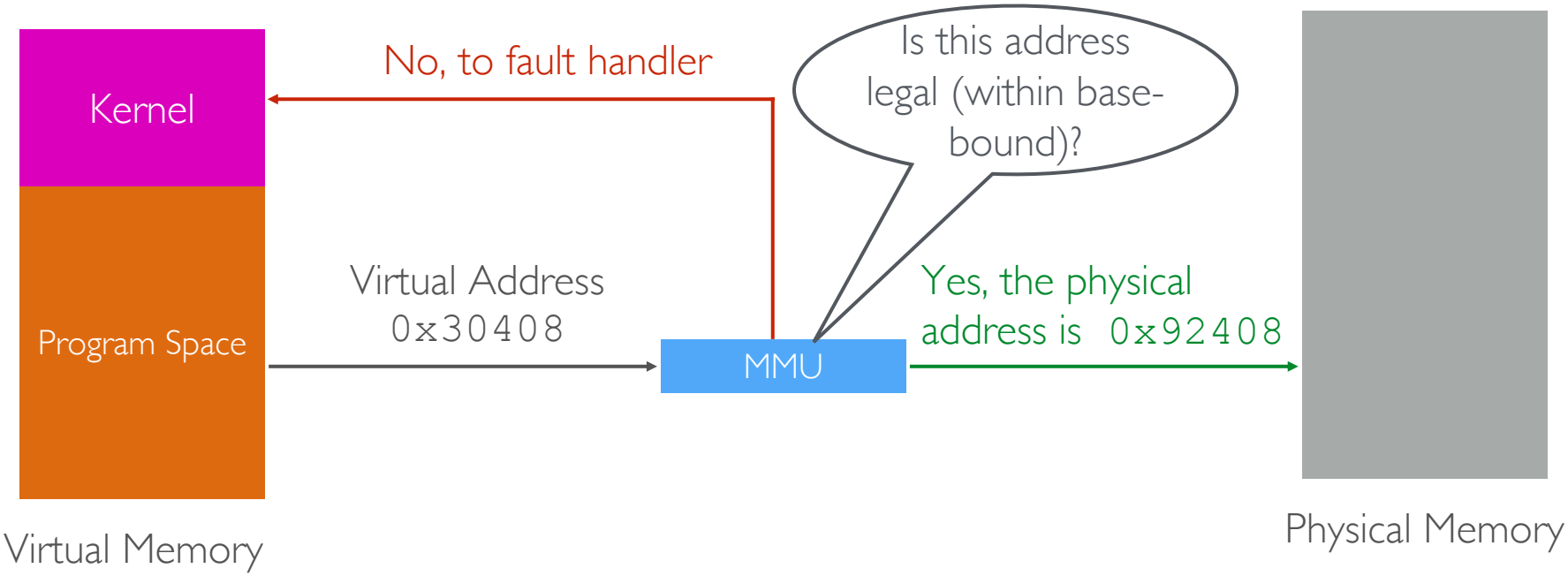
1) User process
2) Hardware
3) OS

What entity should modify the base and bound registers?

1) User process
2) Hardware
3) OS

Hardware support for address translation

Memory-Management Unit (MMU) translates each load/store at runtime



OS management of Base & Bound registers

- ✓ OS decided on location of process when it's created and sets base and bound accordingly
- ✓ OS can change these registers to move the process in memory
- ✓ OS must save and re-load these registers on context switch

(and save these registers for old process)

Base + Bound Trade-offs

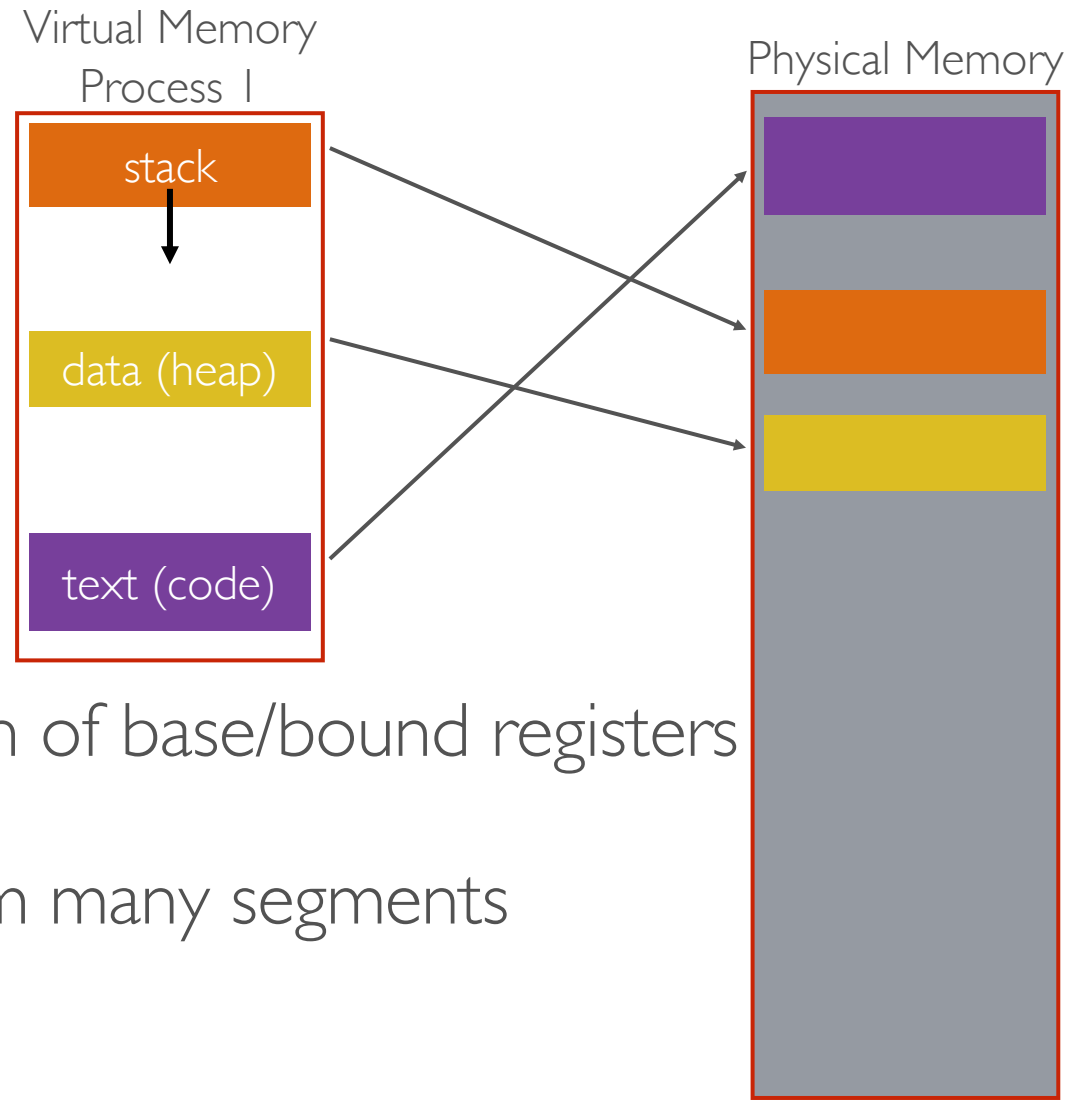
Advantages

- ✓ Cheap in terms of hardware : only two registers
- ✓ Cheap in terms of cycles : do add and compare in parallel
- ✓ Flexible: Can relocate processes

Disadvantages

- ◉ Internal fragmentation (unused wasted space inside process' space)

Attempt 2: Segmentation

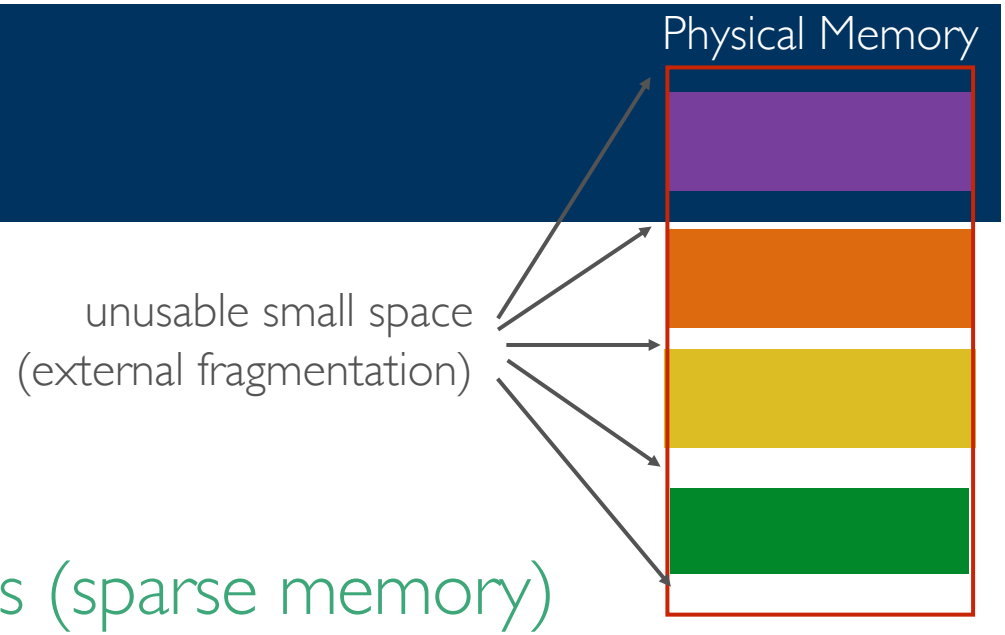


Each process has a collection of base/bound registers

Address space is built from many segments (a.k.a segmentation table)

Each segment has its own base and bound

Segmentation Trade-offs



Advantages

- ✓ Multiple segments per process (sparse memory)
- ✓ Do not need entire process in memory (swap)

Disadvantages

- Makes **external fragmentation** a real problem (unused space between allocated segments)
- Expensive if segment grows larger than expected

Fragmentation

Fragmentation is the inability to use free memory

Over time

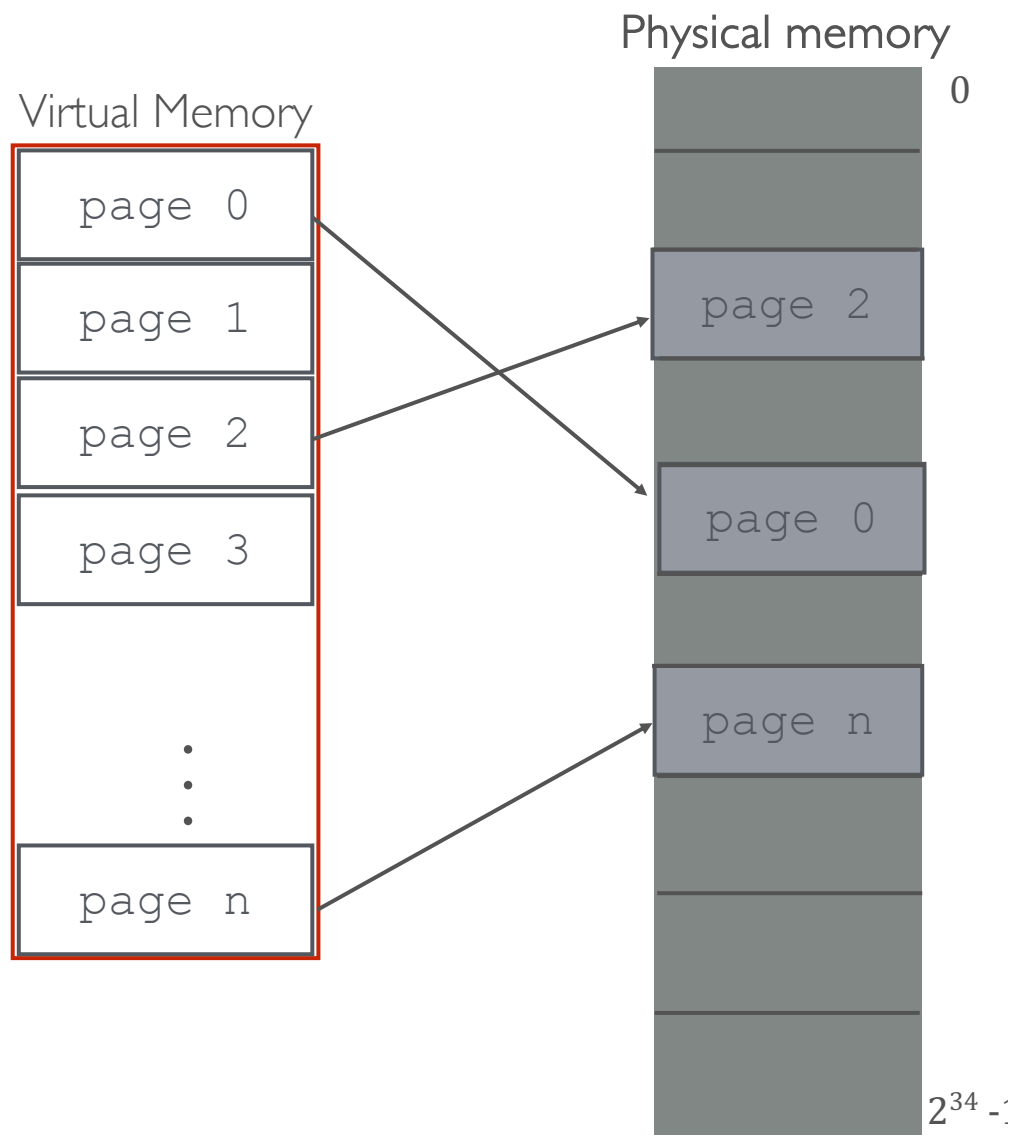
- **External fragmentation**
  because of variable sized pieces (i.e many small holes)
- **Internal fragmentation**
  because of fixed size pieces (i.e no external hole but internal waste of space)

Attempt 3: Simple **Paging**

Divide virtual memory up into fixed-size **pages**

Divide physical memory up into page-sized **page frames** (physical pages)

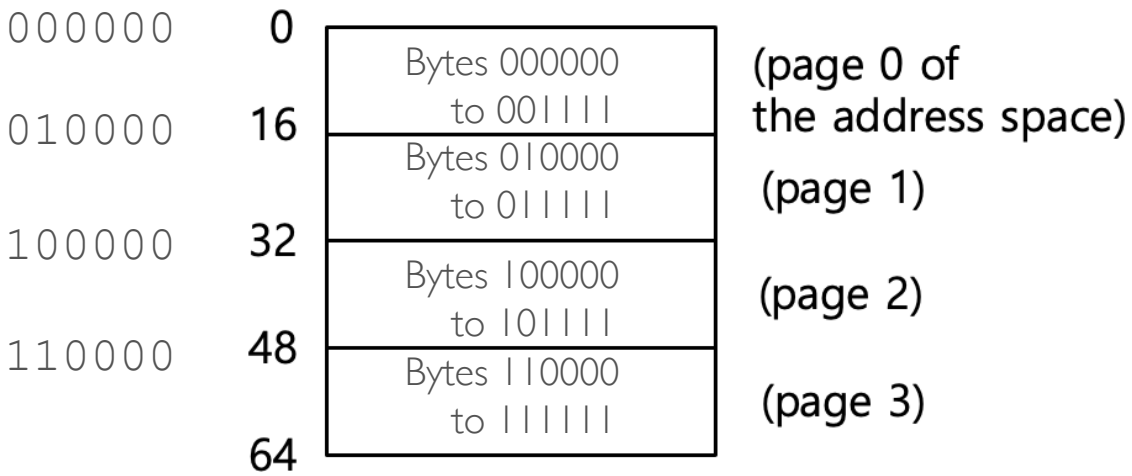
Eliminates external fragmentation



Each process needs a mapping from virtual pages to physical pages (**page table**).

Example: Simple paging

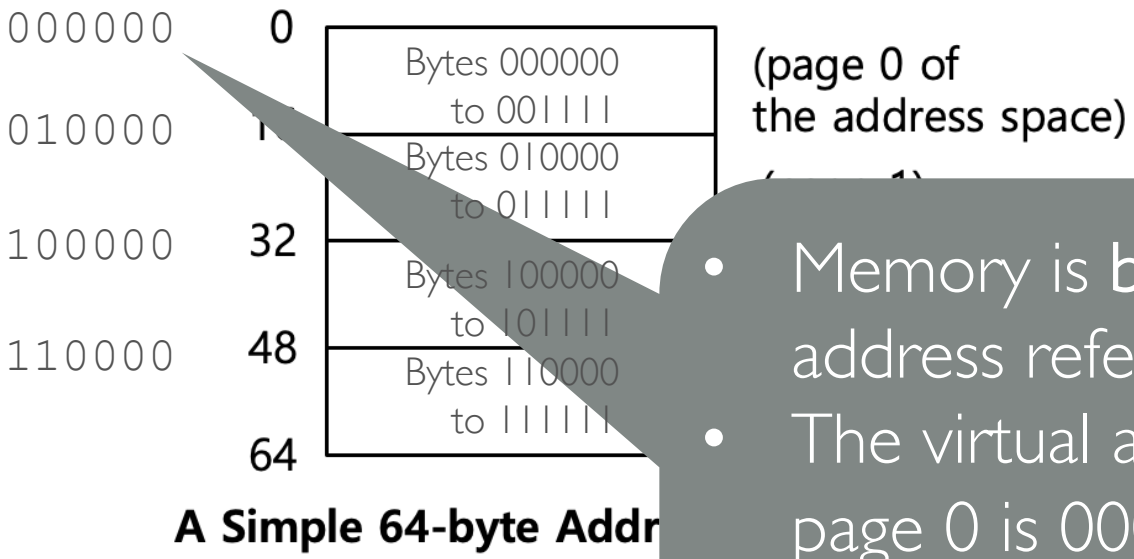
- 128-byte physical memory with 16 byte page frames
- 64-byte address space with 16 byte pages



A Simple 64-byte Address Space

Example: Simple paging

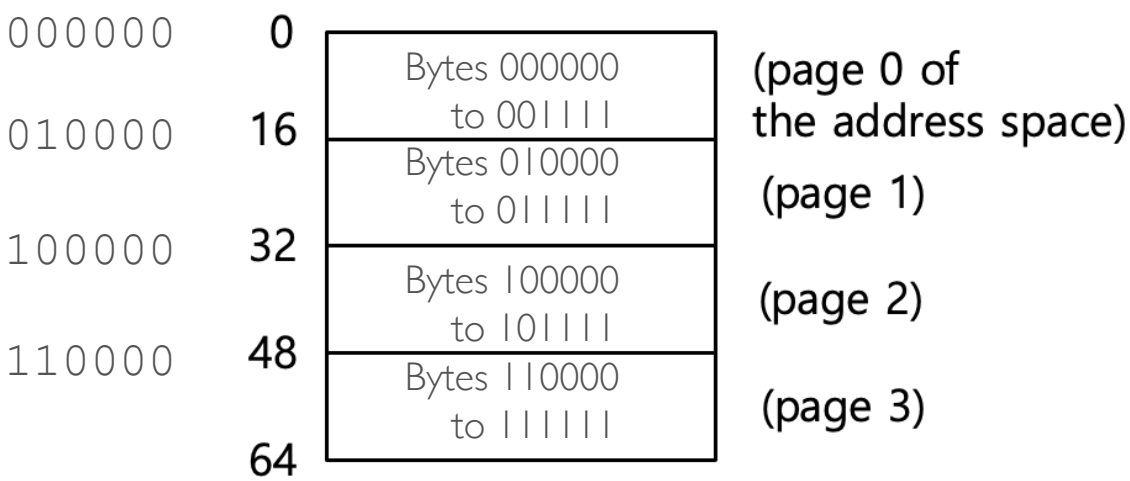
- 128-byte physical memory with 16 byte page frames
- 64-byte address space with 16 byte pages



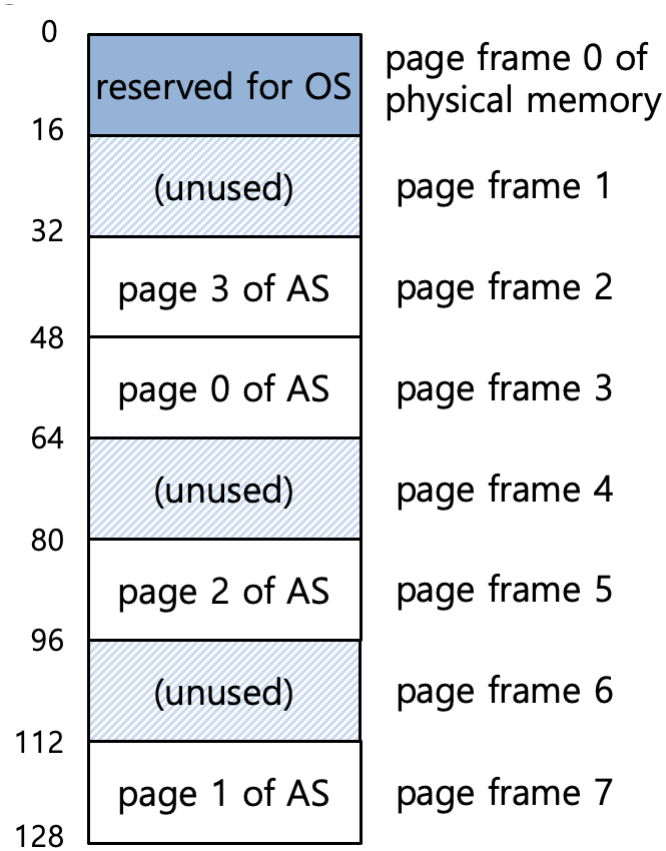
- Memory is **byte-addressable**, i.e. each address refers to one byte.
- The virtual address of the first byte on page 0 is 00000.
- The virtual address of the first byte on page 1 is 010000.
- And so on…

Example: Simple paging

- 128-byte physical memory with 16 byte page frames
- 64-byte address space with 16 byte pages



A Simple 64-byte Address Space



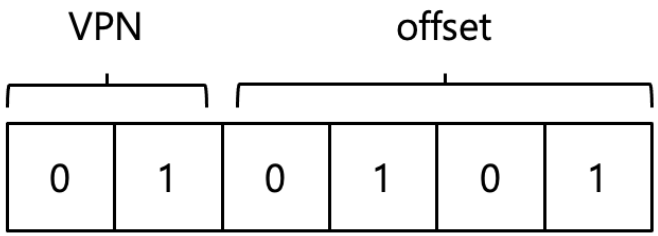
Physical memory

Mapping from virtual to physical addresses

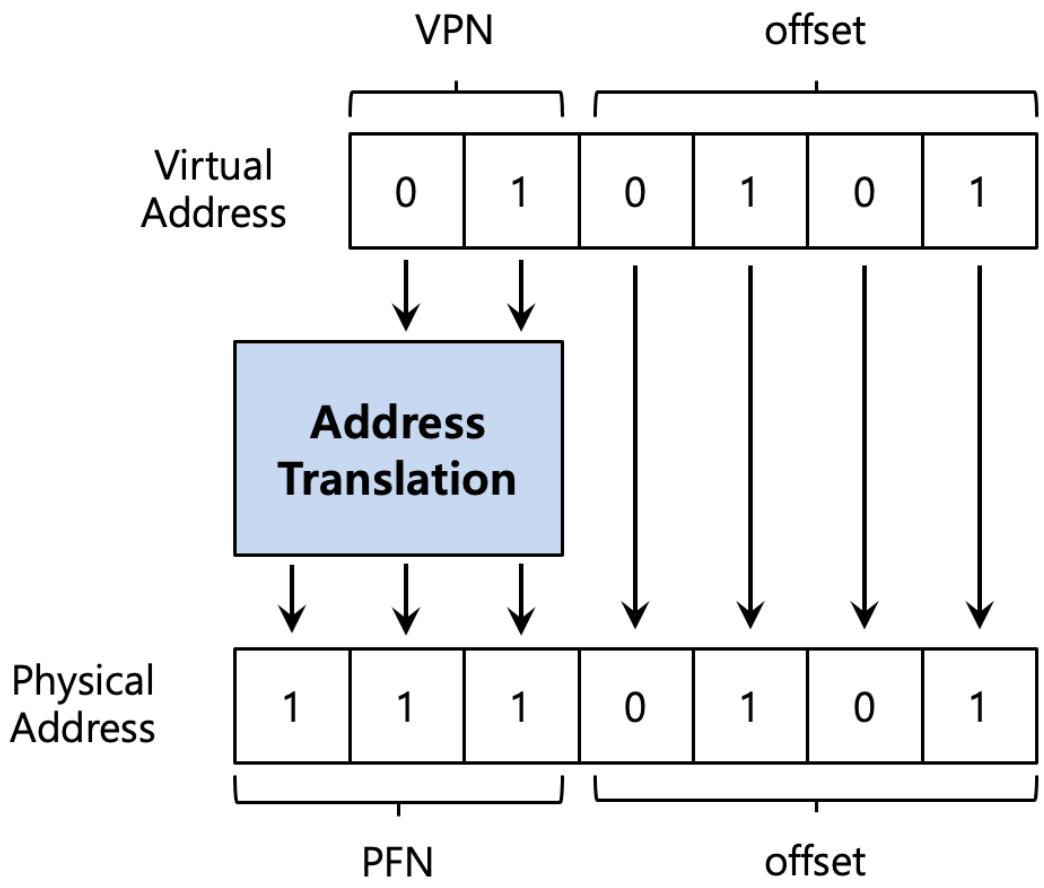
00****	=> page frame 3
01****	=> page frame 7
10****	=> page frame 5
11****	=> page frame 2

Address translation

- Example: virtual address 21 in 64-byte address space



- Two components in the virtual address
  - VPN: virtual page number
  - Offset: offset within the page

Address translation

- The virtual address 21 in 64-byte address space